

СОВРЕМЕННЫЕ ШИФРЫ С СЕКРЕТНЫМ КЛЮЧОМ

Рассмотрим вычислительно стойкие шифры с секретным ключом, которые могут быть вскрыты, но требуют для этого очень большого количества вычислений (например, 1020 лет для суперкомпьютера).

Эти шифры обеспечивают шифрование и дешифрование данных со скоростями, значительно превышающими скорости шифров с открытыми ключами и теоретически стойких шифров, что и объясняет их широкое практическое использование.

Слово **ПЕРЕМЕНА** после применения к нему шифра Цезаря превращается в **ТИУИПИРГ** (если исключить букву Ё и считать, что в алфавите 32 буквы).

Мы можем описать их шифр в общем виде, если пронумеруем (закодируем) буквы русского алфавита числами от 0 до 31 (исключив букву Ё). Тогда правило шифрования запишется следующим образом:

$$c = (m + k) \bmod 32, \quad (1.1)$$

где m и c — номера букв соответственно сообщения и шифротекста, а k — некоторое целое число, называемое ключом шифра (в рассмотренном выше шифре Цезаря $k = 3$).

Чтобы дешифровать зашифрованный текст, нужно применить «обратный» алгоритм:

$$m = (c - k) \bmod 32. \quad (1.2)$$

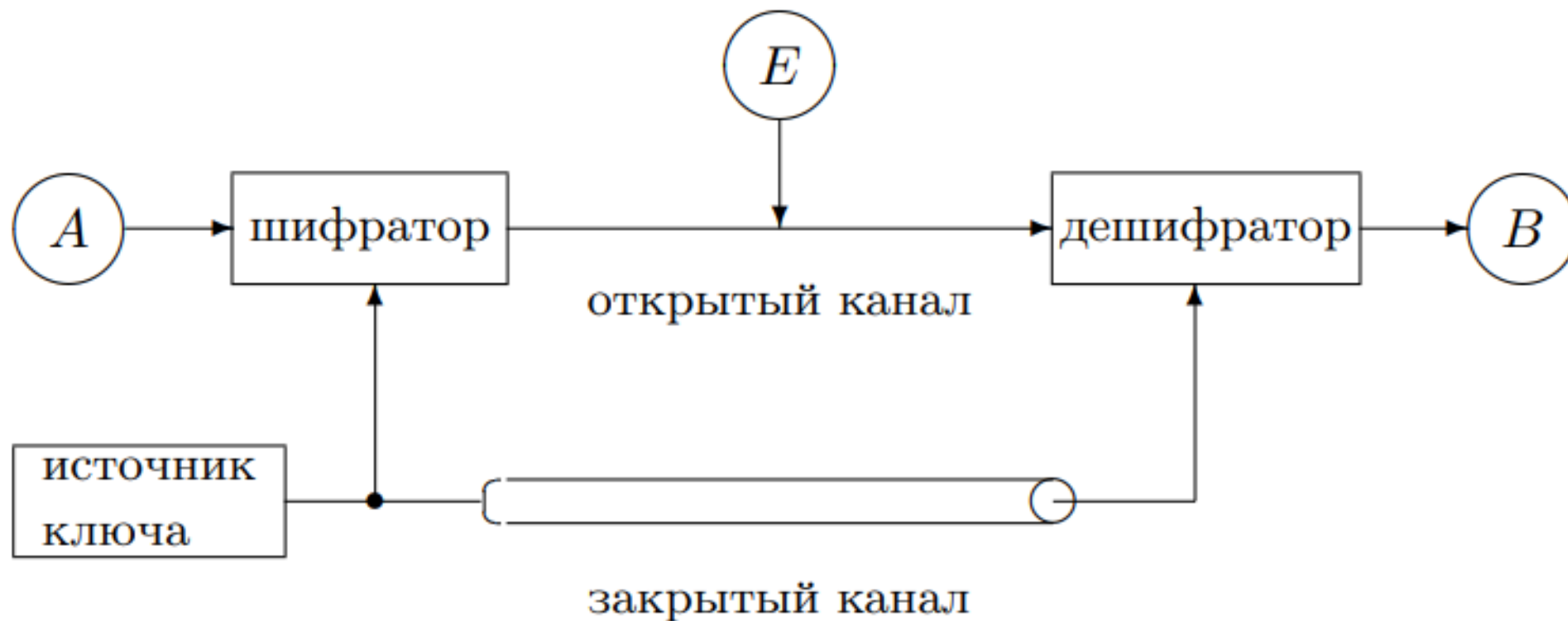


Рис. 1.1. Классическая система секретной связи

Виды атак

- **По шифротексту.**
- **По известному тексту.** Это происходит, если Ева получает в свое распоряжение какие-либо открытые тексты, соответствующие ранее переданным зашифрованным. Сопоставляя пары «текст–шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту.** Противник пользуется не только предоставленными ему парами «текст–шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

Попытки взлома шифра

В нашем примере Ева знает, что шифр был построен в соответствии с (1.1), что исходное сообщение было на русском языке и что был передан шифротекст **ТИУИПИРГ**, но ключ Еве не известен.

Наиболее очевидная попытка расшифровки — последовательный перебор всех возможных ключей (это так называемый метод «грубой силы» - brute-force attack).

Ева перебирает последовательно все возможные ключи:

$k = 1, 2, \dots, 32$, подставляя их в алгоритм дешифрования и оценивая получающиеся результаты.

Т а б л и ц а 1.1. Расшифровка слова ТИУИПИРГ путем перебора ключей

<i>k</i>	<i>m</i>	<i>k</i>	<i>m</i>	<i>k</i>	<i>m</i>	<i>k</i>	<i>m</i>
1	СЗТ	9	ЙЯ	17	БЧ	25	ЩП
2	РЖС	10	ИЮЙ	18	АЦБ	26	ШОЩ
3	ПЕРЕМЕНА	11	ЗЭИ	19	ЯХА	27	ЧН
4	ОДП	12	ЖЪ	20	ЮФ	28	ЦМ
5	НГ	13	ЕЫ	21	ЭУ	29	ХЛЦ
6	МВ	14	ДЪ	22	Ь	30	ФК
7	ЛБМ	15	ГЩ	23	Ы	31	УЙ
8	КАЛАЗ	16	ВШГ	24	Ъ	32	ТИУИПИРГ

Видим, что был использован ключ $k = 3$ и зашифровано сообщение **ПЕРЕМЕНА**.

Очевидно, что рассмотренный шифр совершенно нестойк, для его вскрытия достаточно проанализировать несколько первых букв сообщения и после этого ключ k однозначно определяется (и, следовательно, однозначно дешифруется все сообщение).

В чем же причины нестойкости рассмотренного шифра и как можно было бы увеличить его стойкость?

Рассмотрим еще один пример. Алиса спрятала важные документы в ячейке камеры хранения, снабженной пятидекадным кодовым замком.

Теперь она хотела бы сообщить Бобу комбинацию цифр, открывающую ячейку. Она решила использовать аналог шифра Цезаря, адаптированный к алфавиту, состоящему из десятичных цифр:

$$c = (m + k) \bmod 10. \quad (1.3)$$

Допустим, Алиса послала Бобу шифротекст **26047**. Ева пытается расшифровать его, последовательно перебирая все возможные ключи. Результаты ее попыток сведены в табл. 1.2.

**Т а б л и ц а 1.2. Расшифровка сообщения
26047 путем перебора ключей**

k	m	k	m
1	15936	6	60481
2	04825	7	59370
3	93714	8	48269
4	82603	9	37158
5	71592	0	26047

Все полученные варианты равнозначны и Ева не может понять, какая именно комбинация истинна.

Анализируя шифротекст, она не может найти значения секретного ключа.

Конечно, до перехвата сообщения у Евы было 10^5 возможных значений кодовой комбинации, а после — только 10.

Однако важно отметить то, что в данном случае всего 10 значений ключа. Поэтому при таком ключе (одна десятичная цифра) Алиса и Боб и не могли рассчитывать на большую секретность.

В случае избыточных сообщений шифр легко вскрывается путем перебора ключей.

Для повышения стойкости шифра Цезаря: самое простое, что приходит в голову — увеличить количество возможных значений ключа.

В этом случае Еве придется перебирать больше ключей, прежде чем она найдет единственный правильный.

Естественный способ увеличить количество возможных значений ключа для шифра Цезаря — использовать разные ключи для разных букв сообщения.

Например, мы можем шифровать каждую нечетную букву ключом k_1 , а четную ключом k_2 .

Тогда секретный ключ $k = (k_1, k_2)$ будет состоять из двух чисел, и количество возможных ключей будет $32^2 = 1024$.

Зашифруем наше прежнее сообщение из первой главы ключом $k = (3, 5)$:

$$\text{ПЕРЕМЕНА} \xrightarrow{3,5} \text{ТКУКПКРЕ}. \quad (8.1)$$

Эта схема легко обобщается на произвольную длину секретного ключа $k = (k_1, k_2, \dots, k_t)_{32}$. При t порядка 10 и выше задача полного перебора ключей становится практически нерешаемой.

Данный шифр легко вскрывается путем **частотного криптоанализа**.

При использовании частотного криптоанализа перебор начинают с ключей, соответствующих наиболее часто встречающимся буквам и их сочетаниям.

Известно, что в типичном тексте на русском языке буква **О** встречается чаще других. Смотрим на **ТКУКПКРЕ** и определяем, какие буквы встречаются чаще других на четных и нечетных местах.

На четных местах это **К**. Предполагаем, что это код **О**, следовательно,

$$k_2 = K(11) - O(15) = 28 \pmod{32}.$$

На нечетных местах все буквы различны, поэтому для поиска k_1 знание частот букв языка не помогает (т.к. взято очень короткое сообщение).

Пытаемся найти k_1 путем перебора, но убеждаемся, что приемлемых расшифровок не получается. **Это означает, что наша гипотеза о том, что О заменяется в шифре на К, неверна.**

Берем вместо О другую часто встречающуюся букву — букву Е. Вычисляем $k_2 = K(11) - E(6) = 5$. Повторяем аналогичные действия для поиска k_1 и на этот раз находим решение $k_1 = 3$.

В результате, чтобы расшифровать сообщение, из всех возможных 1024 ключей нам понадобилось проверить только 36:

$$(0, 28), \dots, (31, 28), (0, 5), \dots, (3, 5).$$

Усложним шифр, чтобы затруднить частотный криптоанализ.

Нам нужно каким-то образом перемешать символы сообщения, заставить их влиять друг на друга, чтобы скрыть индивидуальные частоты их появления. По-прежнему будем использовать ключ $k = (k_1, k_2)$ и шифровать сообщение блоками по два символа m_i, m_{i+1} .

Один из простейших вариантов шифра может выглядеть так:

$$\begin{aligned}\tilde{m}_i &= m_i + m_{i+1}, \\ \tilde{m}_{i+1} &= m_{i+1} + \tilde{m}_i, \\ c_i &= \tilde{m}_i + k_1, \\ c_{i+1} &= \tilde{m}_{i+1} + k_2 \quad (\text{mod } 32)\end{aligned}\tag{8.2}$$

(все суммы вычисляются по модулю 32). Здесь m_i — нечетная буква исходного текста, m_{i+1} — четная буква, k_1 , k_2 — символы ключа, а c_i , c_{i+1} — получаемые символы шифротекста.

Например, пара символов ПЕ шифруется ключом $k = (3, 5)$ следующим образом:

$$\begin{aligned}\tilde{m}_i &= \text{П} + \text{Е} = \Phi, \\ \tilde{m}_{i+1} &= \text{Е} + \Phi = \text{Щ}, \\ c_i &= \Phi + 3 = \text{Ч}, \\ c_{i+1} &= \text{Щ} + 5 = \text{Ю},\end{aligned}$$

т.е. ПЕ превращается в ЧЮ.

Этот шифр можно дешифровать. Алгоритм дешифрования, называемый обычно обратным шифром, выглядит следующим образом:

$$\begin{aligned}\tilde{m}_{i+1} &= c_{i+1} - k_2, \\ \tilde{m}_i &= c_i - k_1, \\ m_{i+1} &= \tilde{m}_{i+1} - \tilde{m}_i, \\ m_i &= \tilde{m}_i - m_{i+1} \pmod{32}.\end{aligned}\tag{8.3}$$

Применяя к нашему сообщению шифр (8.2) с ключом (3, 5), получаем:

$$\text{ПЕРЕМЕНА} \longrightarrow \text{ФЩХЪСЦНН} \xrightarrow{3,5} \text{ЧЮШЯФЫРТ}. \quad (8.4)$$

Данный шифр скрывает частоты появления отдельных символов, затрудняя частотный анализ. Конечно, он сохраняет частоты появления пар символов, но мы можем скрыть и их, если будем шифровать сообщения блоками по три символа и т.д.

Виды атак

- **По шифротексту.**
- **По известному тексту.** Это происходит, если Ева получает в свое распоряжение какие-либо открытые тексты, соответствующие ранее переданным зашифрованным. Сопоставляя пары «текст–шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту.** Противник пользуется не только предоставленными ему парами «текст–шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

Что произойдет, если Ева каким-то образом достала открытый текст, соответствующий ранее переданному зашифрованному сообщению? (Атака по известному тексту).

Например, Ева имеет пару (ПЕРЕМЕНА, ТКУКПКРЕ) для шифра (8.1).

Тогда она сразу вычисляет секретный ключ

$$k_1 = T - П = 3 ,$$

$k_2 = К - Е = 5$, и легко расшифровывает все последующие сообщения от Алисы к Бобу.

При использовании шифра (8.2) пара (ПЕРЕМЕНА, ЧЮШЯФЫРТ) уже не дает такого очевидного решения, хотя и здесь оно довольно просто.

Ева применяет две первые операции из (8.2) (не требующие секретного ключа) к слову ПЕРЕМЕНА, получает промежуточный результат ФЩХЪСЦНН и уже по паре (ФЩХЪСЦНН, ЧЮШЯФЫРТ), как и в первом случае, находит ключ $k = 3, 5$.

Как затруднить такие действия Евы?

Будем при шифровании сообщения использовать шифр (8.2) два раза. Тогда получим:

$$\text{ПЕРЕМЕНА} \xrightarrow{3,5} \text{ЧЮШЯФЫРТ} \xrightarrow{3,5} \text{ШШЪЫТПЕЩ}. \quad (8.5)$$

Имея пару (**ПЕРЕМЕНА**, **ШШЪЫТПЕЩ**), Ева **не может вычислить ключ** (она не может получить промежуточное значение ЧЮШЯФЫРТ, так как при его построении был использован секретный ключ, ей не известный).

В схеме (8.5) отдельная реализация алгоритма (8.2) называется **раундом** или **циклом** шифра.

На стойкость шифра влияют: длина ключа, размер блока, количество раундов.

Для повышения стойкости необходимо введения «перемешивающих» преобразований.

В реальных шифрах используются те же преобразования, но над более длинными цепочками символов и обладающие целым рядом дополнительных свойств.

Блочные шифры

Блочный шифр можно определить как зависящее от ключа K обратимое преобразование слова X из n двоичных символов. Преобразованное с помощью шифра слово (или блок) будем обозначать через Y . Для всех рассматриваемых в этом разделе шифров длина слова Y равна длине слова X .

Таким образом, блочный шифр — это обратимая функция E (другим словами, такая, для которой существует обратная функция). Конкретный вид E_K этой функции определяется ключом K ,

$$\begin{aligned} Y &= E_K(X), \\ X &= E_K^{-1}(Y) \quad \text{для всех } X. \end{aligned}$$

Здесь E_K^{-1} обозначает дешифрующее преобразование и называется *обратным шифром*.

Для криптографических приложений блочный шифр должен удовлетворять ряду требований, зависящих от ситуации, в которой он используется. В большинстве случаев достаточно потребовать, чтобы шифр был стоек по отношению к атаке по выбранному тексту. Это автоматически подразумевает его стойкость по отношению к атакам по шифротексту и по известному тексту. Следует заметить, что при атаке по выбранному тексту шифр всегда может быть взломан путем перебора ключей. Поэтому требование стойкости шифра можно уточнить следующим образом.

Шифр *считается стойким* (при атаке по выбранному тексту), если для него *неизвестны* алгоритмы взлома, существенно более эффективные, чем прямой перебор ключей.

DES (Data Encryption Standard)

Стандарт США 1977 года.

Параметры: размер блока 64 бита, длина ключа 56 бит, 16 раундов.

В 2001 году после специально объявленного конкурса в США был принят новый стандарт на блочный шифр **AES (Advanced Encryption Standard)**, в основу которого был положен шифр **Rijndael**, разработанный бельгийскими специалистами.

Шифр ГОСТ 28147-89

Стандарт 1989 года.

Параметры: длина ключа 256 бит, размер блока 64 бита, 32 раунда.

Более удобен для программной реализации, чем DES, выигрыш по времени примерно в 1.5 раза.

Величина основных параметров (длина ключа, размер блока, количество раундов) позволяют утверждать, что шифр не может быть слабым. Никаких эффективных атак против шифра ГОСТ 28147-89 не опубликовано.

В основе ГОСТ 28147-89, так же как и DES, лежит так называемая структура Фейстела. Блок разбивается на две одинаковые части, правую R и левую L . Правая часть объединяется с ключевым элементом и посредством некоторого алгоритма шифрует левую часть. Перед следующим раундом левая и правая части меняются местами. Такая структура позволяет использовать один и тот же алгоритм как для шифрования, так и для дешифрования блока. Это особенно важно при аппаратной реализации, так как прямой и обратный шифры формируются одним и тем же устройством (различается только порядок подачи элементов ключа).

Перейдем к непосредственному описанию шифра ГОСТ 28147-89. Введем необходимые определения и обозначения. Последовательность из 32 бит будем называть словом. Блок текста X (64 бита), также как блок шифротекста Y , состоит из двух слов — левого L и правого R , причем L — старшее слово, а R — младшее. Секретный ключ K (256 бит) рассматривается состоящим из восьми слов $K = K_0 K_1 \cdots K_7$. На его основе строится так называемый раундовый ключ $W = W_0 W_1 \cdots W_{31}$, состоящий из 32 слов (метод построения раундового ключа будет дан позже).

Для работы шифра нужны 8 таблиц $S_0, S_1, \dots, S_7$ (называемых также S-блоками). Каждая таблица содержит 16 четырехбитовых элементов, нумеруемых с 0 по 15. Будем обозначать через $S_i[j]$ j -й элемент i -й таблицы. ГОСТ рекомендует заполнять каждую таблицу различными числами из множества $\{0, 1, \dots, 15\}$, переставленными случайным образом. Содержимое таблиц формирует дополнительный секретный параметр шифра, общий для большой группы пользователей. Заметим, что в DES аналогичные S-блоки фиксированы и несекретны.

В шифре используются следующие операции:

- $+$ сложение слов по модулю 2^{32} ;
- $\leftarrow$ циклический сдвиг слова влево на указанное число бит;
- $\oplus$ побитовое «исключающее или» двух слов, т.е. побитовое сложение по модулю 2.

Алгоритм 8.1. БАЗОВЫЙ ЦИКЛ ШИФРА ГОСТ 28147-89

ВХОД: Блок L, R , раундовый ключ W .

ВЫХОД: Преобразованный блок L, R .

1. FOR $i = 0, 1, \dots, 31$ DO
2. $k \leftarrow R + W_i, \quad k = (k_7 \cdots k_0)_{16};$
3. FOR $j = 0, 1, \dots, 7$ DO
4. $k_j \leftarrow S_j[k_j];$
5. $L \leftarrow L \oplus (k \leftarrow 11);$
6. $L \longleftrightarrow R;$
7. RETURN L, R .

(На шаге 4 алгоритма используются отдельные четырехбитовые элементы переменной k .)

С помощью базового цикла осуществляется шифрование и дешифрование блока. Чтобы зашифровать блок, строим раундовый ключ

$$W = K_0 K_1 K_2 K_3 K_4 K_5 K_6 K_7 K_0 K_1 K_2 K_3 K_4 K_5 K_6 K_7 \\ K_0 K_1 K_2 K_3 K_4 K_5 K_6 K_7 K_7 K_6 K_5 K_4 K_3 K_2 K_1 K_0, \quad (8.6)$$

подаем на вход X и на выходе получаем Y .

Чтобы дешифровать блок, строим раундовый ключ

$$W = K_0 K_1 K_2 K_3 K_4 K_5 K_6 K_7 K_7 K_6 K_5 K_4 K_3 K_2 K_1 K_0 \\ K_7 K_6 K_5 K_4 K_3 K_2 K_1 K_0 K_7 K_6 K_5 K_4 K_3 K_2 K_1 K_0, \quad (8.7)$$

подаем на вход Y и на выходе получаем X .

Программная реализация обычно требует переработки цикла 3 алгоритма 8.1, так как работа с полубайтами неэффективна. Ясно, что то же самое преобразование может быть выполнено с использованием четырех таблиц по 256 байт или двух таблиц по 65536 полуслов. Например, при работе с байтами имеем $k = (k_3 \cdots k_0)_{256}$, и шаги 3—4 алгоритма 8.1 переписываются следующим образом:

- 3. FOR $j = 0, 1, 2, 3$ DO
- 4. $k_j \leftarrow T_j[k_j]$.

Таблицы T_j , $j = 0, 1, 2, 3$, вычисляются предварительно из S-боксов:

```
FOR  $i = 0, 1, \dots, 255$  DO  
     $T_j[i] \leftarrow S_{2j}[i \bmod 16] + 16S_{2j+1}[i \operatorname{div} 16]$ .
```

ГОСТ ничего не говорит о правилах формирования ключевой информации, кроме требования, чтобы каждый S-блок содержал перестановку различных чисел. В то же время ясно, что, например, нулевой ключ или тривиально заданные S-боксы (отображающие k в себя) не обеспечивают секретности шифра.

Шифр RC6

Создан Рональдом Ривестом в 1998 году.

В RC6 пользователь задает **размер слова** (w) 16, 32 или 64 бита, **количество раундов** (r), **длину ключа** (l) от 0 до 255 байт.

Размер блока всегда составляет четыре слова. Конкретный вариант шифра обозначается по схеме RC6- $w/r/l$.

Например, RC6-32/20/16 — размер блока 128 бит, длина ключа 128 бит, 20 раундов (такой шифр исследовался в качестве кандидата на стандарт США).

В шифре используются следующие операции:

- $+$, $-$ сложение и вычитание слов по модулю 2^w ;
- $*$ умножение по модулю 2^w ;
- $\oplus$ побитовое сложение по модулю 2 или, что то же самое, «исключающее или» двух слов;
- $\leftarrow, \rightarrow$ циклические сдвиги слова влево или вправо на указанное число бит (заметим, что при длине слова w бит величина циклического сдвига фактически приводится по модулю w , причем, как правило, это приведение выполняется автоматически на машинном уровне, т.е. не требует дополнительных вычислений — процессор просто использует младшие $\log w$ бит числа, задающего величину сдвига).

Шифрование и дешифрование блока данных производится с использованием одного и того же раундового ключа W длиной $2r + 4$ слова (нумерация слов с нуля), получаемого из секретного ключа K (метод формирования раундового ключа будет рассмотрен ниже).

Алгоритм 8.2. RC6: ШИФРОВАНИЕ БЛОКА ДАННЫХ

ВХОД: Блок из четырех слов (a, b, c, d) , раундовый ключ W .

ВЫХОД: Зашифрованный блок (a, b, c, d) .

1. $b \leftarrow b + W_0, d \leftarrow d + W_1;$
2. FOR $i = 1, 2, \dots, r$ DO
3. $t \leftarrow (b * (2b + 1)) \leftarrow \log w,$
4. $u \leftarrow (d * (2d + 1)) \leftarrow \log w,$
5. $a \leftarrow ((a \oplus t) \leftarrow u) + W_{2i},$
6. $c \leftarrow ((c \oplus u) \leftarrow t) + W_{2i+1},$
7. $(a, b, c, d) \leftarrow (b, c, d, a);$
8. $a \leftarrow a + W_{2r+2}, c \leftarrow c + W_{2r+3};$
9. RETURN $(a, b, c, d).$

Для дешифрования просто «прокручиваем» процесс в обратном порядке.

Алгоритм 8.3. RC6: ДЕШИФРОВАНИЕ БЛОКА ДАННЫХ

ВХОД: Блок из четырех слов (a, b, c, d) , раундовый ключ W .

ВЫХОД: Дешифрованный блок (a, b, c, d) .

1. $c \leftarrow c - W_{2r+3}, a \leftarrow a - W_{2r+2};$
2. FOR $i = r, r - 1, \dots, 1$ DO
3. $(a, b, c, d) \leftarrow (d, a, b, c),$
4. $t \leftarrow (b * (2b + 1)) \leftrightarrow \log w,$
5. $u \leftarrow (d * (2d + 1)) \leftrightarrow \log w,$
6. $a \leftarrow ((a - W_{2i}) \hookrightarrow u) \oplus t,$
7. $c \leftarrow ((c - W_{2i+1}) \hookrightarrow t) \oplus u;$
8. $d \leftarrow d - W_1, b \leftarrow b - W_0;$
9. RETURN $(a, b, c, d).$

Расписание раундового ключа в RC6 более сложно, чем в ГОСТ 28147-89, что характерно для большинства современных шифров. По сути дела речь идет о разворачивании секретного ключа K в более длинную псевдослучайную последовательность W с целью затруднить криптоанализ шифра.

Обозначим через c число слов в ключе, $c = 8l/w$. Посредством нижеприведенного алгоритма секретный ключ K разворачивается в раундовый ключ W :

$$K_0 K_1 \cdots K_{c-1} \longrightarrow W_0 W_1 \cdots W_{2r+3}.$$

В алгоритме используются «магические» числа: P_w — первые w бит двоичного разложения числа $e - 2$, где e — число Эйлера, служащее основанием натурального логарифма, и Q_w — первые w бит двоичного разложения числа $\phi - 1$, где $\phi = (\sqrt{5} - 1)/2$ — «золотое сечение». В табл. 8.1 значения P_w и Q_w приведены в шестнадцатиричной системе счисления для различных длин слов w .

Т а б л и ц а 8.1. «Магические» числа для RC6

w	16	32	64
P_w	b7e1	b7e15163	b7e15162 8aed2a6b
Q_w	9e37	9e3779b9	9e3779b9 7f4a7c15

Алгоритм 8.4. РС6: ФОРМИРОВАНИЕ РАУНДОВОГО КЛЮЧА

ВХОД: Секретный ключ K .

ВЫХОД: Раундовый ключ W .

1. $W_0 \leftarrow P_w$;
2. FOR $i = 1, 2, \dots, 2r + 3$ DO $W_i \leftarrow W_{i-1} + Q_w$;
3. $a \leftarrow 0, b \leftarrow 0, i \leftarrow 0, j \leftarrow 0$;
4. $k \leftarrow 3 \max(c, 2r + 4)$;
5. DO k раз
6. $W_i \leftarrow (W_i + a + b) \leftrightarrow 3, \quad a \leftarrow W_i$,
7. $K_j \leftarrow (K_j + a + b) \leftrightarrow (a + b), \quad b \leftarrow K_j$,
8. $i \leftarrow i + 1 \bmod 2r + 4, \quad j \leftarrow j + 1 \bmod c$;
9. RETURN W .

Рассмотрим кратко основные идеи построения алгоритма шифрования RC6. Заметим прежде всего, что, как и в шифрах DES и ГОСТ 28147-89, в каждом раунде RC6 одна половина блока используется для шифрования другой. Действительно, значение переменных t и u (строки 3–4 алгоритма 8.2) определяется только словами b и d соответственно. Затем эти переменные используются для модификации слов a и c перед сложением с элементами ключа (строки 5–6). Таким образом, в a и c вносится зависимость от b и d . В следующем раунде пары a, c и b, d меняются ролями, причем b и d переставляются между собой (строка 7 алгоритма). Вследствие такой структуры шифра количество раундов должно быть четным.

Выбранная для вычисления переменных t и u функция вида $f(x) = (2x^2 + x) \bmod 2^w$ характеризуется сильной зависимостью старших бит своего значения от всех бит аргумента. Именно старшие биты f должны определять величину сдвига в строках 5–6 алгоритма. Поэтому необходимое их количество переносится в младшие разряды t и u посредством вращения $\leftarrow \log w$. При модификации a и c использование «исключающего или» достаточно традиционно, тогда как зависимое от данных вращение (операция $\leftarrow$) — характерная особенность RC6 (заимствованная у его более простого предшественника RC5).

Строки 1 и 8 алгоритма 8.2 скрывают значения слов, которые не изменялись в ходе первого и последнего раундов.

Рекомендуемое количество раундов $r = 20$ связано с результатами исследования стойкости шифра по отношению к дифференциальному и линейному криптоанализу.

В ходе исследований шифра слабых ключей не обнаружено, т.е. любой ключ, даже нулевой, обеспечивает заявляемую высокую стойкость шифра. Предполагается, что для RC6 не существует алгоритма взлома, лучшего, чем перебор ключей.